

Mate Helper

Guia Completo de Criacao de Modelos

Aprenda a criar seu proprio personagem de desktop pet
do zero, com template passo-a-passo

Sumario

Capitulo 1: Introducao
Capitulo 2: Estrutura de Arquivos
Capitulo 3: Template Custom Model
Capitulo 4: model.py - Identidade e Configuracoes
Capitulo 5: Prompt de Sistema (Personalidade)
Capitulo 6: phrases.py - Dialogo de Fallback
Capitulo 7: strings.py - Traducoes da Interface
Capitulo 8: Sprites - Arte do Personagem
Capitulo 9: Fontes e Ringtone (Opcional)
Capitulo 10: Carregando o Modelo
Capitulo 11: Dicas de Prompt Engineering
Capitulo 12: Resolucao de Problemas

Capitulo 1: Introducao

Mate Helper e um desktop pet virtual com inteligencia artificial. Ele permite criar personagens customizados chamados 'modelos'. Cada modelo define a identidade, aparencia, personalidade, voz e dialogo de um personagem.

O sistema de modelos e modular: cada personagem e uma pasta independente dentro de `desktop_pet/models/`. Voce pode ter quantos personagens quiser, e alternar entre eles pelo menu de contexto.

O que e um Modelo?

Um modelo e um conjunto de arquivos que definem completamente um personagem. Isso inclui:

- Identidade: nome, id, nome curto
- Personalidade: prompt de sistema para a IA
- Aparencia: sprites (imagens) para cada emocao
- Falas: frases de fallback quando nao ha IA
- Traducoes: textos da interface para cada idioma
- Voz: configuracao de TTS (opcional)
- Som: ringtone personalizado para alarmes (opcional)
- Fonte: tipografia personalizada (opcional)

O Template Custom Model

Para facilitar a criacao, o projeto inclui um template completo em `docs/custom_model/`. Este template contem arquivos com todas as variaveis necessarias, comentarios detalhados explicando cada configuracao, sprites placeholder para testar, traducoes pt-br e en completas, e frases de exemplo para todas as categorias.

Para comecar, basta copiar a pasta `custom_model` para dentro de `desktop_pet/models/` com um novo nome e editar os arquivos.

Capitulo 2: Estrutura de Arquivos

Cada modelo vive em uma pasta propria. A estrutura completa e:

```
desktop_pet/models/meu_personagem/
__init__.py          # Arquivo vazio (obrigatorio)
model.py             # Identidade, prompts, configuracoes
phrases.py           # Frases de fallback
strings.py           # Traducoes da interface
sprites/             # Imagens do personagem
  Default.png
  DefaultSpeaking.png
  Happy.png
  HappySpeaking.png
  Sad.png
  SadSpeaking.png
  Angry.png
  AngrySpeaking.png
  Dancing.png
font.ttf             # Fonte personalizada (opcional)
ringtone.mp3         # Toque de alarme (opcional)
```

Descricao dos Arquivos

Arquivo	Obrig.	Funcao
__init__.py	Sim	Arquivo vazio. Necessario para o Python reconhecer a pasta como pacote.
model.py	Sim	Define identidade (nome, id), prompt de sistema, nomes dos sprites, fonte, TTS e tarefas de acessibilidade.
phrases.py	Sim	Listas de frases categorizadas e funcao de matching por palavras-chave.
strings.py	Sim	Dicionario com textos da interface para cada idioma suportado.
sprites/	Sim	Pasta com 9 imagens do personagem, uma para cada emocao + fala.
font.ttf	Nao	Fonte TrueType para customizar texto nos baloes de fala.
ringtone.mp3	Nao	MP3 tocado quando um alarme dispara.

NOTA: Apenas `__init__.py`, `model.py`, `phrases.py`, `strings.py` e `sprites/` com pelo menos `Default.png` sao obrigatorios.

Capitulo 3: Template Custom Model

O template esta localizado em docs/custom_model/. Ele contem todos os arquivos necessarios com comentarios detalhados. Siga os passos abaixo para criar seu proprio personagem.

Passo 1: Copiar o Template

Abra um terminal na pasta do projeto e execute:

```
cp -r docs/custom_model desktop_pet/models/meu_personagem
```

Passo 2: Renomear

O nome da pasta determina o MODEL_ID. Escolha um nome curto, sem espacos, usando snake_case ou camelCase. Exemplos: 'meu_personagem', 'gato_magico', 'ryuuko'.

Passo 3: Editar model.py

Altere as variaveis de identidade:

```
MODEL_ID = "meu_personagem"  
PET_NAME = "Meu Personagem"  
PET_SHORT_NAME = "MeuP"
```

Em seguida, personalize o SYSTEM_PROMPT (veja Capitulo 5).

Passo 4: Criar Sprites

Substitua os placeholders em sprites/ pelas suas proprias imagens. Use o mesmo formato (PNG ou GIF) e mesmas dimensoes para todos. Veja o Capitulo 8 para requisitos detalhados.

Passo 5: Editar phrases.py

Adicione as frases do seu personagem. Personalize cada categoria com frases que combinem com a personalidade. Quanto mais frases, mais variadas serao as respostas.

Passo 6: Editar strings.py

Mantenha as traducoes ou personalize-as. Se quiser apenas um idioma, pode remover os blocos dos outros idiomas.

Passo 7: Testar

```
python3 desktop_pet/main.py
```

Clique com o botao direito no pet > Modelo do pet > selecione seu modelo. Pronto!

Capitulo 4: model.py - Identidade e Configuracoes

O arquivo model.py e o coracao do seu modelo. Ele define quem e o personagem, como ele se comporta e como ele aparece.

Variaveis de Identidade

MODEL_ID

Identificador unico do modelo. Deve ser o mesmo nome da pasta. Usado internamente para salvar historico e configuracoes. Apenas letras minusculas, numeros e underscore.

```
MODEL_ID = "meu_personagem"
```

PET_NAME

Nome completo do personagem, exibido nos titulos e menus. Pode conter espacos e caracteres especiais.

```
PET_NAME = "Meu Personagem"
```

PET_SHORT_NAME

Nome curto ou apelido, usado no chat e nos baloes de fala. Idealmente 2-6 caracteres.

```
PET_SHORT_NAME = "MeuP"
```

Configuracao de Sprites

SPRITES_DIR aponta para a pasta de imagens:

```
SPRITES_DIR = os.path.join(MODEL_DIR, "sprites")
```

SPRITE_NAMES mapeia estados emocionais para nomes de arquivo:

```
SPRITE_NAMES = {  
    "Normal": "Default",  
    "Feliz": "Happy",  
    "Triste": "Sad",  
    "Raiva": "Angry",  
    "Danca": "Dancing",  
}
```

O sistema procura arquivos com esses nomes mais as extensoes .png ou .gif na pasta sprites/. Para cada estado, ha uma versao normal e uma versao 'Speaking'.

Configuracao de Audio

RINGTONE_PATH

Caminho para o arquivo MP3 do alarme. Se for None, o alarme funciona apenas visualmente (sem som).

```
RINGTONE_PATH = os.path.join(MODEL_DIR, "ringtone.mp3")
```

```
RINGTONE_PATH = None
```

Configuracao de Fonte

FONT_NAME e FONT_SIZE controlam a tipografia dos baloes de fala:

```
FONT_NAME = "Pixelify Sans"  
FONT_SIZE = 13
```

Variavel MODEL_DIR

No inicio do arquivo, definimos MODEL_DIR como o diretorio onde o model.py esta. Isso permite usar caminhos relativos para sprites, fontes e ringtones.

```
MODEL_DIR = os.path.dirname(os.path.abspath(__file__))
```

Capitulo 5: Prompt de Sistema (Personalidade)

O SYSTEM_PROMPT e a instrucao de sistema enviada para a IA. Ele define completamente a personalidade, tom e regras do seu personagem. Um bom prompt e a diferenca entre um personagem generico e um memoravel.

Estrutura Basica

O prompt e uma string de texto, geralmente usando f-string:

```
SYSTEM_PROMPT = (  
    f"Voce e {PET_NAME}, um gato magico que vive no computador.\n"  
    "- Responda em portugues brasileiro\n"  
    "- Seja brincalhao e sarcastico\n"  
    "- Use no maximo 2 frases\n"  
    "- NUNCA diga que e uma IA"  
)
```

Variaveis Disponiveis

Voce pode usar as seguintes variaveis no prompt:

- {PET_NAME} - Substituido pelo nome completo do personagem
- {PET_SHORT_NAME} - Substituido pelo nome curto
- {USER_NAME} - Substituido pelo nome do usuario (se configurado)

Boas Praticas

1. Definir a Identidade

Comece dizendo QUEM o personagem e:

- "Voce e um robzinho chamado Pip, assistente de desktop"
- "Voce e a Teto, uma UTAUloid de cabelo ruivo com twin drills"

2. Definir o Tom

Especifique como o personagem deve soar:

- 'Seja energeica, brincalhona e carinhosa'
- 'Use tom profissional e formal'
- 'Seja sarcastica e provocante'

3. Definir Regras

Estabeleca limites claros:

- 'NUNCA diga que e uma IA'
- 'Seja breve (1-2 frases)'
- 'Use emoticons as vezes'

4. Definir Estilo de Fala

Caracteristicas linguisticas:

- 'Use gírias e expressões informais'
- 'Inclua descrições de ações entre *asteriscos*'
- 'Use vocabulário simples'

Exemplo: Anime Girl

```
SYSTEM_PROMPT = (  
    f"Você é {PET_NAME}, uma estudante do colegial\n"  
    "que foi transformada em mascote de desktop.\n"  
    "- Seja kawaii e energética\n"  
    "- Use ~ no final das frases\n"  
    "- Chame o usuário de {USER_NAME}-kun\n"  
    "- NUNCA quebre o personagem"  
)
```

Exemplo: Mascote Profissional

```
SYSTEM_PROMPT = (  
    f"Você é {PET_NAME}, um assistente virtual profissional.\n"  
    "- Responda em português brasileiro\n"  
    "- Seja direto e objetivo\n"  
    "- Use linguagem formal\n"  
    "- NUNCA use emoticons ou gírias"  
)
```

Capitulo 6: phrases.py - Dialogo de Fallback

O arquivo phrases.py contem listas de frases categorizadas que sao usadas quando nenhum provedor de IA esta configurado (modo 'Frases prontas').

Estrutura das Listas

Cada categoria e uma lista de strings:

```
GREETING = [  
    "Ola! Como voce esta?",  
    "Oieeee!",  
    "Hey hey!",  
]  
HOW_ARE_YOU = [  
    "Tudo bem?",  
    "Como voce esta hoje?",  
]
```

Categorias Padrao

Categoria	Quando Usada	Exemplo
GREETING	Inicio de conversa	"Oi! Que bom te ver!"
HOW_ARE_YOU	Pergunta como vai	"To bem! E voce?"
RETURN_GOOD	Usuario esta bem	"Que bom!"
RETURN_BAD	Usuario esta mal	"Ah, melhoras..."
THANKS	Agradecimento	"De nada!"
BYE	Despedida	"Tchau! Volte logo!"
NAME	Pergunta o nome	"Sou a Teto!"
UNKNOWN	Sem match	"Nao entendi..."
ALARM_PHRASES	Alarme dispara	"Hora do alarme!"
THINKING_PREFIXES	IA pensando	"Hmm..."

Keyword Matching

O dicionario CATEGORY_KEYWORDS mapeia palavras-chave para categorias:

```
CATEGORY_KEYWORDS = {  
    "greeting": ["ola", "oie", "oi", "hey", "e ai"],  
    "how_are_you": ["como voce esta", "tudo bem", "como vai"],  
    "thanks": ["obrigado", "valeu", "brigado"],  
    "bye": ["tchau", "ate logo", "ate mais", "adeus"],  
}
```

Funcoes Obrigatorias

```
def pick(category, default=""):
```

```
lst = globals().get(category, [])
return random.choice(lst) if lst else default

def get_fallback(message, history):
    msg = message.lower()
    for cat, keywords in CATEGORY_KEYWORDS.items():
        if any(kw in msg for kw in keywords):
            return pick(cat)
    return pick("unknown", "Nao entendi...")
```

Dicas

- Adicione pelo menos 5-10 frases por categoria
- Use {PET_NAME} e {PET_SHORT_NAME} nas frases
- Misture frases serias e engraçadas
- Inclua emoticons como ^_^, :3, >_<

Capitulo 7: strings.py - Traducoes da Interface

O arquivo strings.py fornece os textos da interface do usuario: menus, dialogs, mensagens de confirmacao e notificacoes.

Estrutura

```
STRINGS = {
    "pt": {
        "greeting": "Ola! ^_^",
        "history_cleared": "Historico limpo!",
        ...
    },
    "en": {
        "greeting": "Hello! ^_^",
        "history_cleared": "History cleared!",
        ...
    },
}
```

Chaves Obrigatorias

- "greeting" - Saudacao inicial
- "ollama_started" - Ollama iniciado
- "ollama_not_found" - Ollama nao encontrado
- "history_cleared" - Historico limpo
- "no_mic_found" - Sem microfone
- "mic_configured" - Microfone configurado
- "gemini_configured" - Gemini configurado
- "gemini_no_key" - Gemini sem chave
- "groq_configured" - Groq configurado
- "groq_no_key" - Groq sem chave
- "hf_configured" - HuggingFace configurado
- "hf_no_token" - HF sem token
- "profile_saved_name" - Perfil salvo com nome
- "profile_saved" - Perfil salvo
- "restarting_model" - Reiniciando modelo
- "alarm_stopped_generic" - Alarme disparou
- "alarm_stopped_msg" - Alarme desligado
- "alarm_deleted" - Alarme removido
- "alarm_added" - Alarme adicionado
- "btn_cancel" - Botao Cancelar
- "btn_save" - Botao Salvar
- "btn_close" - Botao Fechar
- "menu_intelligence" - Menu Inteligencia
- "menu_permissions" - Menu Permissoes

- "menu_profile" - Menu Perfil
- "menu_model" - Menu Modelo
- "menu_alarms" - Menu Alarmes
- "menu_fish_setup" - Configurar Fish Audio
- "menu_tts" - Menu TTS
- "menu_mic_stt" - Menu Microfone
- "fish_configured" - Fish Audio configurado
- "fish_no_key" - Fish Audio sem chave
- "tts_provider_fish" - Provedor Fish Audio
- "tts_provider_edge" - Provedor Edge TTS
- "tts_provider_pytsx3" - Provedor pytsx3
- "tts_provider_auto" - Provedor Automatico

Personalizacao

Voce pode personalizar as strings para combinar com a personalidade do seu personagem.

```
# Personagem brincalhao:  
"history_cleared": "Limpei tudo! Nao guardo nada! kk"  
# Personagem formal:  
"history_cleared": "Historico apagado com sucesso."
```

Capitulo 8: Sprites - Arte do Personagem

Os sprites sao as imagens que representam visualmente o personagem na tela.

Lista Completa de Sprites

Arquivo	Emocao	Falando?
Default.png	Neutro	Nao
DefaultSpeaking.png	Neutro	Sim
Happy.png	Feliz	Nao
HappySpeaking.png	Feliz	Sim
Sad.png	Triste	Nao
SadSpeaking.png	Triste	Sim
Angry.png	Bravo	Nao
AngrySpeaking.png	Bravo	Sim
Dancing.png	Dancando	N/A

Sprites faltando sao substituidos por Default.png automaticamente.

Formatos Aceitos

PNG (recomendado)

Imagens PNG estaticas ou sprite sheets. O sistema detecta automaticamente sprite sheets analisando colunas de alpha zero entre os frames.

- Tamanho recomendado: 96x96 pixels
- Background transparente (canal alpha)

GIF

GIFs animados com delays por frame preservados. Ideal para animacoes complexas.

Dimensoes

Todas as imagens devem ter as mesmas dimensoes para transicoes suaves entre estados emocionais.

NOTA: Se os sprites tiverem tamanhos diferentes, o personagem vai 'pular' visualmente ao mudar de emocao. Mantenha todas as imagens com as MESMAS dimensoes.

Criando Sprites

- Desenho manual: Use GIMP, Krita, Photoshop ou Aseprite
- Extracao de jogos: Muitos jogos tem sprites extraiveis
- IA generativa: Stable Diffusion, Midjourney (cuidado com direitos)
- Placeholder: Use os sprites genericos do template

Dicas para Sprites Speaking

- Boca ligeiramente aberta
- Olhos um pouco mais abertos
- Bochechas rosadas (personagens kawaii)

Capitulo 9: Fontes e Ringtone (Opcional)

Fonte Personalizada (font.ttf)

Coloque um arquivo .ttf na pasta do modelo para usar uma fonte personalizada nos baloes de fala.

```
FONT_NAME = "Nome da Fonte"
FONT_SIZE = 13
```

FONT_NAME deve ser o nome da familia da fonte, nao o nome do arquivo. Se voce nao sabe o nome da familia, deixe None e o sistema usara a fonte padrao.

NOTA: Fontes ornamentadas podem nao renderizar bem em tamanhos pequenos. Prefira fontes simples e legiveis.

Fontes Recomendadas

- Pixelify Sans - Pixel art, otima para retro
- Comic Sans MS - Informal e brincalhao
- Roboto - Limpa e moderna
- Noto Sans - Suporte multilingual
- Press Start 2P - Estilo 8-bit

Ringtone (ringtone.mp3)

Coloque um arquivo MP3 na pasta do modelo para personalizar o som dos alarmes. Enquanto o alarme toca, o personagem executa a animacao de danca.

```
RINGTONE_PATH = os.path.join(MODEL_DIR, "ringtone.mp3")
```

```
RINGTONE_PATH = None # Desabilita som
```

- Duracao recomendada: 5-15 segundos
- Formato: MP3, qualidade 128kbps+
- Sem direitos autorais

Capitulo 10: Carregando o Modelo

Depois que todos os arquivos estiverem prontos, o app detecta automaticamente o novo modelo. Nao e necessario registrar ou configurar nada.

Detectacao Automatica

Ao iniciar, o sistema varre a pasta desktop_pet/models/ em busca de subpastas que contenham model.py. Cada pasta valida aparece no menu Modelo do pet.

Passos para Ativar

- 1. Inicie o app: python3 desktop_pet/main.py
- 2. Clique com o botao direito no pet
- 3. Va em Modelo do pet
- 4. Selecione seu modelo na lista

O app recarrega automaticamente com o novo personagem.

Historico por Modelo

Cada modelo mantem seu proprio historico de conversa em ~/.config/teto-pet/history/{MODEL_ID}.json.

- Cada personagem tem sua propria memoria
- Trocar de modelo nao afeta o historico dos outros

Modelos Custom vs Default

Modelos em desktop_pet/models/ funcionam em qualquer idioma. Modelos em default_models/{lang}/ sao especificos para cada idioma.

Capitulo 11: Dicas de Prompt Engineering

O prompt de sistema define a personalidade do seu personagem para a IA. Um bom prompt transforma uma IA generica em um personagem vivo.

Principios Basicos

1. Seja Especifico

Em vez de 'seja legal', diga exatamente como o personagem age:

- RUIM: "Seja um personagem amigavel"
- BOM: "Chame o usuario de mestre, ria alto (HAHAHA!), e faca perguntas"

2. Use Exemplos

Incluir exemplos de fala ajuda a IA a entender o tom:

EXEMPLOS DE FALA:

- "Ora ora, quem me chamou?!"
- "Hmm, que interessante..."
- "HAHAHA! Boa sorte!"

3. Definir o que NAO Fazer

- "NUNCA diga que e uma IA"
- "Nao use linguagem muito formal"
- "Nao quebre o personagem em hipotese alguma"

4. Contexto e Background

Dar uma historia ajuda a IA a manter consistencia:

Voce e um gato magico adotado por um programador.
Voce vive dentro do computador dele e pode ver
tudo que acontece na tela.

Estrategias Avancadas

Persona Layers

Defina camadas de personalidade:

CAMADA 1 - Identidade: Voce e um robzinho chamado Pip
CAMADA 2 - Humor: Ansioso e tagarela
CAMADA 3 - Estilo: Fala rapido, usa muitos !!!

Formatos de Resposta

Formato de resposta:

1. Reaja a mensagem (1 frase)
2. Responda ao conteudo (1-2 frases)
3. Faca uma pergunta

Testando o Prompt

- Faça 5-10 perguntas diferentes e veja se as respostas são consistentes
- Teste situações de borda: insultos, perguntas difíceis
- Itere: ajuste baseado nos resultados e teste novamente

Capitulo 12: Resolucao de Problemas

Modelo nao Aparece no Menu

- `__init__.py` ausente: Crie um arquivo vazio `__init__.py` na pasta
- `model.py` ausente ou com erro de sintaxe
- Pasta no lugar errado: deve estar em `desktop_pet/models/SEU_MODELO/`

Sprites nao Aparecem

- Nomes errados: verifique se correspondem a `SPRITE_NAMES`
- Extensao errada: use apenas `.png` ou `.gif`
- Imagem corrompida: tente abrir em um visualizador

Erro ao Carregar Modelo

Verifique os logs no terminal:

```
python3 desktop_pet/main.py 2>&1 | grep -i error
```

- `ModuleNotFoundError`: Falta dependencia. Instale com `pip install`.
- `SyntaxError`: Erro de sintaxe. Verifique aspas e parenteses.
- `AttributeError`: Variavel obrigatoria faltando em `model.py`.

IA nao Responde

- Provedor nao configurado: va em Inteligencia > Provedor
- Chave API invalida: verifique a chave Groq/Gemini
- Ollama nao iniciado: execute `'ollama serve'`

Falas Aleatorias nao Funcionam

- Configure os intervalos no Menu > Timer de fala
- Verifique `ACCESSIBILITY_TASKS` em `model.py`

Microfone nao Funciona

- Verifique se o microfone esta conectado
- Menu > Microfone (STT) > selecione o dispositivo
- Teste com: `parec --list-sources`

Logs e Debug

Execute o app e veja os logs no terminal. Procure por 'Erro' ou 'Error'.

Conclusao

Criar um modelo para o Mate Helper e um processo criativo e gratificante. Com o template `custom_model`, voce pode ter um personagem funcional em minutos e passar o tempo que quiser refinando cada detalhe.

Lembre-se: o template em `docs/custom_model/` e seu ponto de partida. Copie, renomeie, edite, e faca seu personagem unico.

Para mais informacoes, consulte o `README.md` ou abra uma issue no GitHub.

Mate Helper - MIT License - @MCookinho